



Interview Task: Caching Strategy & Consistency (.NET)

Space: Engineering Hiring Version: 1.0 Last Updated: April 2026

- interview-task
- senior-level
- backend
- .NET 8



Table of Contents

- General Information
- Goal of the Assignment
- Business Scenario (Minimal & Fixed)
- Part 1 – Caching Layer (MANDATORY)
- Part 2 – Cache Invalidation (MANDATORY)
- Part 3 – Expiration Strategy (MANDATORY)
- Part 4 – Advanced Requirement (MANDATORY)
- Explicitly Out of Scope
- Submission
- Submission

General Information

This assignment evaluates your architectural and design skills. Business logic is intentionally simple.

Item	Details
Level	Senior .NET Developer
Focus	Dependency Injection, Service Lifetimes, Concurrency & State
Tech Stack	.NET 8, ASP.NET Core Web API

Allowed Tools	Documentation, Google, Stack Overflow, AI tools
Important	You must justify caching decisions

Goal of the Assignment

This assignment evaluates your ability to:

- Design a proper caching layer
- Avoid common cache bugs
- Handle cache invalidation
- Prevent stale data issues
- Understand memory vs distributed cache
- Apply correct lifetime & expiration strategy

Business logic is simple.

Architecture & correctness are the focus.

Business Scenario (Minimal & Fixed)

You are implementing a Product Catalog API.

Endpoints:

```
GET  /api/products/{id}
POST /api/products
PUT  /api/products/{id}
```

Products are stored in an in-memory repository (Dictionary).

Goal: Improve read performance using caching.

Part 1 – Caching Layer (MANDATORY)

You must implement:

- A caching abstraction (e.g., IProductCache)
- Use IMemoryCache OR custom abstraction
- Apply caching only for GET

Requirements:

- Cache key must be deterministic
- Cache expiration strategy must be defined
- Avoid caching null accidentally (your design decision)

Part 2 – Cache Invalidation (MANDATORY)

You must handle:

- Product update
- Product creation

Cache must:

- Invalidate or refresh correctly
- Avoid stale reads
- Not require application restart

Demonstrate correctness via simple test scenario.

Part 3 – Expiration Strategy (MANDATORY)

You must implement at least one:

- Absolute expiration
- Sliding expiration

You must justify choice during defense.

Part 4 – Advanced Requirement (MANDATORY)

Add one of the following:

Option A:

- Cache stampede prevention (double-check locking or GetOrCreateAsync pattern)

Option B:

- Simulate distributed cache via interface abstraction

Option C:

- Implement manual cache invalidation endpoint

Choose one and be ready to justify.

Explicitly Out of Scope

Do NOT implement:

- Redis (optional abstraction only)
- Database
- Third-party caching libraries
- Complex persistence
- Background refresh services

Focus on fundamentals.

Submission

Zip project folder.

README must include ONLY:

- How to run
- Example request flow
- Example demonstrating cache hit/miss

No design explanation required.

Submission

Submit a link to a GitHub repository.

- The repository must be **public** (or shared with the interviewer)
- README must include: how to run, example request/response, brief notes on key design decisions
- No lengthy design explanation required — architectural decisions are covered in the oral defense